

NetSigorta

Sunum Soru-Cevap Hazırlığı

Mimari · Katmanlar · Veritabanı · Alternatif Teknolojiler

Bölüm A — Mimari mevcut durum

Katman	Proje	Sorumluluk
Sunum	Kasko.API	Denetleyiciler, JWT, rol yetkisi, CORS, Swagger, hata ara katmanı, arka plan servisi
İş	Kasko.Business	Servisler, fiyat motoru, PDF (QuestPDF), DTO'lar, FluentValidation, istisnalar
Veri erişimi	Kasko.DataAccess	EF Core DbContext, generic + özel repository'ler, Unit of Work, migration'lar, audit
Varlık	Kasko.Entities	BaseEntity (Id, Created/Updated/Deleted tarih ve kullanıcı, IsDeleted), enum'lar
Ortak	Kasko.Shared	Paylaşılan yardımcılar
Test	Kasko.Business.Tests	210 birim testi (xUnit + Moq)
İstemci	Angular 22	Standalone bileşenler, signals, zoneless, interceptor'lar, rol guard'ları

Bağımlılık yönü tek taraflıdır: API → Business → DataAccess → Entities. Üst katman alt katmanı arayüzler üzerinden kullanır; bu sayede servisler Moq ile test edilebilir.

Bölüm B — 20 soru ve cevap

1. [Mimari] Neden katmanlı (N-Tier) mimari seçtiniz?

Proje ekibi ve süre için en anlaşılır ve sürdürülebilir yapı. Sorumluluklar ayrık: denetleyici yalnızca HTTP ile, servis iş kuralıyla, repository veriyle ilgilenir. İş kuralları API'den bağımsız olduğu için birim testleri veritabanı olmadan çalışır.

2. [Mimari] Bunun yerine hangi mimariler kullanılabilirdi?

Clean / Onion Architecture: Domain merkezde, altyapı dışta; bağımlılıklar içe doğru. **CQRS + MediatR:** Okuma ve yazma komutları ayrı işleyicilerde; dashboard gibi ağır okumalar ayrı modelle optimize edilir. **Vertical Slice:** Her özellik (Teklif Oluştur, Poliçe Yenile) kendi klasöründe uçtan uca. **Mikroservis:** Teklif, Poliçe, Ödeme, Bildirim ayrı servisler; bu ölçekte gereksiz karmaşıklık getirirdi.

3. [Mimari] Neden mikroservis değil de monolit?

Tek ekip, tek veritabanı ve güçlü tutarlılık gereken bir satış akışı (teklif → ödeme → poliçe) var. Mikroservis dağıtık transaction, servisler arası iletişim ve ayrı dağıtım maliyeti getirir. Katmanlı modüler monolit ileride Ödeme veya Bildirim gibi parçaları ayrı servise çıkarmaya uygundur.

4. [Mimari] Repository ve Unit of Work deseni neden var? EF Core zaten bunu yapmıyor mu?

EF Core DbContext zaten bir Unit of Work, DbSet de repository gibidir. Biz yine de soyutladık çünkü: servisler IUnitOfWork arayüzüne bağımlı olduğundan Moq ile kolayca test ediliyor; teklif + snapshot + ödeme + poliçe gibi çok tablolu işlemler tek SaveChanges ile atomik kaydediliyor. Alternatif olarak servisler doğrudan DbContext kullanıp testlerde InMemory/SQLite sağlayıcısı kullanılabilirdi.

5. [Mimari] Dependency Injection nasıl kullanılıyor?

ASP.NET Core'un yerleşik DI konteyneri. Servis ve repository'ler Scoped (istek başına bir örnek), ExpirationWorker ise Hosted Service olarak kayıtlı. Alternatif: Autofac veya Scrutor ile otomatik tarama.

6. [Mimari] Hata yönetimi nasıl?

Merkezi ExceptionMiddleware. İş katmanı BadRequestException / NotFoundException fırlatır; ara katman bunları uygun HTTP koduna ve Türkçe mesaja çevirir. Frontend'de errorInterceptor mesajı bildirim olarak gösterir. Alternatif: .NET 8 IExceptionHandler + ProblemDetails standardı veya Result deseni (istisna yerine sonuç nesnesi).

7. [Veritabanı] Neden SQL Server ve ilişkisel veritabanı?

Sigorta verisi doğası gereği ilişkilidir: Müşteri → Araç → Teklif → Poliçe → Ödeme. Parasal tutarlar, transaction ve yabancı anahtar bütünlüğü gerekir. SQL Server, EF Core ve Azure SQL ile doğal uyumludur. Alternatifler: PostgreSQL (açık kaynak, maliyetsiz), MySQL; belge veritabanı (MongoDB) yalnızca log veya PDF arşivi gibi yan veriler için uygun olurdu.

8. [Veritabanı] Veri modelinizi anlatır mısınız?

Ana tablolar: Users, Roles, Customers, Vehicles, VehicleValueCatalogs (TSB, 79.380 kayıt), InsurancePackages, Coverages, CoverageOptions, PackageCoverages (paket-teminat çoka-çok), Quotes, QuoteCoverages, QuotePricingSnapshots, Policies, Payments, PricingRules, PricingRuleChangeRequests, TariffChangeRequests, PolicyCancellationRequests, Notifications, BrandSettings. Birincil anahtarlar GUID.

9. [Veritabanı] Code First mi Database First mi? Şema nasıl yönetiliyor?

Code First. Varlık sınıflarından EF Core migration'ları üretiliyor (Add-Migration / Update-Database). Her şema değişikliği sürüm kontrolünde bir migration dosyası olarak duruyor. Canlıya ise SSMS ile bacpac aktarımı yapıldı. Alternatif: canlıda migration'ları CI/CD hattında otomatik uygulamak veya DbUp / Flyway gibi SQL betik tabanlı araçlar.

10. [Veritabanı] Silinen kayıtlar ne oluyor?

Soft delete: kayıt fiziksel silinmez; IsDeleted, DeletedDate ve DeletedBy işaretlenir. Sigortacılıkta geçmiş kayıt yasal olarak saklanmalıdır. Müşteri silinince teklifleri ve araçları da pasife alınır; aktif poliçeli müşteri silinemez. İyileştirme: EF Core global query filter ile silinmiş kayıtları otomatik gizlemek.

11. [Veritabanı] Kim neyi ne zaman değiştirdi, izleniyor mu?

Evet. SaveChanges içinde audit alanları otomatik dolar: CreatedDate/By, UpdatedDate/By, DeletedDate/By. Fiyat kuralları ayrıca sürümlüdür (v1, v2...) ve değişiklik talepleri gerekçe ve karar notuyla saklanır. Alternatif: SQL Server Temporal Tables ile satır geçmişini veritabanı seviyesinde tutmak.

12. [Veritabanı] Tarife değişince eski teklifin fiyatı değişir mi?

Hayır. Her teklif oluşturulduğunda kullanılan katsayılar QuotePricingSnapshots tablosuna yazılır. Geçerlilik süresi boyunca fiyat sabittir; bu "snapshot" desenidir.

13. [Veritabanı] Eşzamanlılık (iki kişi aynı kaydı değiştirirse) nasıl çözülüyor?

Polisede RowVersion ile iyimser eşzamanlılık (optimistic concurrency) kullanılıyor; çakışmada ikinci kayıt reddedilir. Ödeme + poliçe oluşturma tek transaction içinde. Aynı araca çıkışan aktif poliçe iş kuralıyla engelleniyor.

14. [Veritabanı] Performans için ne yaptınız, ne yapılabilir?

TSB kataloğu marka/tip/yıl üzerinden sorgulanıyor; listeler sayfalıyor. İyileştirmeler: sık sorgulanan kolonlara (PlateNumber, IdentityNumber, Status, CustomerId) index; salt okunur sorgularda AsNoTracking; dashboard için önceden hesaplanmış özet tablo veya view; Redis ile katalog ve tarife ön belleği.

15. [Güvenlik] Kimlik doğrulama ve yetkilendirme nasıl?

JWT Bearer token; rol claim'i ile [Authorize(Roles=...)] denetimi. Şifreler ASP.NET Core PasswordHasher ile hash'leniyor. Müşteri yalnızca kendi kayıtlarını görür (servis katmanında sahiplik kontrolü). Alternatifler: ASP.NET Core Identity, refresh token, Azure AD B2C veya Keycloak ile harici kimlik sağlayıcı.

16. [Güvenlik] Gizli bilgiler (şifre, anahtar) nerede tutuluyor?

Yerelde .NET User Secrets; canlıda Azure App Service ortam değişkenleri. appsettings dosyalarında şifre yok. İleri seviye: Azure Key Vault ve Managed Identity ile veritabanına şifresiz bağlantı.

17. [Fiyatlama] Fiyat motoru nasıl tasarlandı, neden ayrı?

PricingService tek sorumluluklu bir servis: kuralları sürüm ve tarihe göre okur, katsayıları çarpar, teminat eklerini hesaplar ve tüm ara değerleri döndürür. Hızlı teklif, portal teklifi ve simülatör aynı motoru kullanır; bu yüzden sonuçlar her kanalda aynıdır. Alternatif: kuralları Strategy deseniyle ayrı sınıflara bölmek veya bir kural motoru (NRules) kullanmak.

18. [Frontend] Frontend mimarisi nasıl?

Angular 22, standalone bileşenler, signals ile reaktif durum, zoneless değişiklik algılama. Özellik bazlı klasörleme (features/quotes, policies...), ortak tema SCSS, HTTP interceptor'lar (token ekleme, hata, yükleme animasyonu), rol guard'ları. Ortam dosyalarıyla yerel/canlı API adresi ayrılıyor. Alternatifler: React + Redux/Zustand, Vue + Pinia, Angular'da NgRx ile merkezi durum yönetimi.

19. [Altyapı] Canlı ortam mimarisi nedir, başka nasıl kurulabilirdi?

Azure Static Web Apps (Angular), App Service B1 (API), Azure SQL Basic (veritabanı), aynı bölgede. Alternatifler: Docker imajı + Azure Container Apps veya AKS; tek App Service altında API + Angular; GitHub Actions ile otomatik CI/CD; Application Insights ile izleme; Azure Front Door ile CDN ve özel alan adı.

20. [Kalite] Test stratejiniz nedir, eksik olan ne?

210 birim testi (xUnit + Moq): fiyat motoru, teklif durum geçişleri, poliçe, iptal, kullanıcı ve müşteri servisleri. Ayrı bir entegrasyon test projesi de var. Eklenebilecekler: Testcontainers ile gerçek SQL Server üzerinde entegrasyon testleri, Playwright/Cypress ile uçtan uca arayüz testleri, k6 ile yük testi.

Bölüm C — Alternatif teknoloji özeti

Alan	Bizde	Alternatifler
Mimari	Katmanlı (N-Tier)	Clean/Onion, CQRS + MediatR, Vertical Slice, Mikroservis
ORM	EF Core 8	Dapper (hızlı ham SQL), NHibernate
Veritabanı	SQL Server / Azure SQL	PostgreSQL, MySQL; MongoDB (log/arşiv), Redis (önbellek)
Doğrulama	FluentValidation	DataAnnotations
Kimlik	JWT + PasswordHasher	ASP.NET Core Identity, Azure AD B2C, Keycloak, refresh token
PDF	QuestPDF	iText, DinkToPdf, Syncfusion
Arka plan işi	BackgroundService (30 dk)	Hangfire, Quartz.NET, Azure Functions timer
Mesajlaşma	Yok (senkron)	RabbitMQ, Azure Service Bus (bildirim/e-posta kuyruğu)
Loglama	Yerleşik ILogger	Serilog + Seq, Application Insights
Frontend	Angular 22 + signals	React, Vue, NgRx
Dağıtım	Kudu ZipDeploy + SWA CLI	GitHub Actions, Azure DevOps, Docker + Container Apps
Test	xUnit + Moq	NUnit, NSubstitute, Testcontainers, Playwright

Kısa cevap ipuçları

- Soruya önce bizdeki çözümle, sonra "alternatif olarak..." diye cevap verin; neden seçmediğinizi bir cümleyle söyleyin.
- Bilmediğiniz bir teknolojiye: "Bu ölçekte gerekmediği için tercih etmedik, ileride şu noktada eklenebilir."
- Her cevabı mümkünse ekranda gösterin: simülatör, kural geçmişi, Swagger, canlı adres.